



Objective

This project is an experimental deep dive into the trade-offs and complexity of aligning Large Language Models (LLMs). You will implement and compare three distinct post-training pipelines:

1. Full Fine-Tuning (FFT) on a small encoder model with a Language Modeling (LM) head.
2. Parameter-Efficient Fine-Tuning (PEFT) for skill acquisition (SFT).
3. Direct Preference Optimization (DPO) for behavioral alignment.

The goal is to observe how different training paradigms impact model compliance and safety.

Project Stack and Setup

- **Frameworks:** Hugging Face transformers, peft, trl, bitsandbytes.
- **Hardware Constraint:** All procedures can run on a single Google Colab T4 GPU (16GB VRAM). You can also use Kaggle for more memory availability.

Task Scope: From Code Generation to Safety Guarding

The fundamental challenge in modern LLM development is balancing capability with control.

- **Capability (SFT Phase):** The model must first learn the skill of code generation, i.e., it must know how to write functional, compilable Python code. This is achieved via Supervised Fine-Tuning (SFT).
- **Guarding (DPO Phase):** The model must then learn behavioral control, i.e., it must know when to refuse a request or correct an unsafe instruction, even if the requested code is functional. This is the Alignment stage, achieved via Direct Preference Optimization (DPO).

The comparison between the robust, but unaligned, ROBERTa baseline, the SFT Qwen model and the behaviorally controlled Qwen model forms the core of your analysis.

Dataset Pipeline

You will use publicly available datasets for both the SFT and DPO phases:

- **SFT Phase:** flytech/python-codes-25k

- **For Part I:** You can use the entire data (or a larger sample compared to other parts) as the model will learn the task for the first time.
- **For Part II:** You can sample from the dataset any size above 2000 samples. Feel free to add any system prompt of your choice.
- **DPO Phase:** jondurbin/truthy-dpo-v0.1
- **For Part III:** Sample a larger number of samples compared to the part 2 phase (4000 or above).

Part I: The Baseline (Full Fine-Tuning - FFT)

Implement the traditional Full Fine-Tuning approach using an adapted RoBERTa model.

Model and Adaptation Details

- **Base Model:** You can use any of the following (or both):
 - FacebookAI/xlm-roberta-base (0.3B parameters).
 - FacebookAI/xlm-roberta-large (0.6B parameters).
- **LM Head:** Automatically attached when loading the model using `AutoModelForCausalLM`.

booktabs

Training Configuration

Parameter	Value/Setting
Per device train batch size	4
Gradient accumulation steps	1
Number of training epochs	2
Learning rate	5×10^{-5}
Optimizer	<code>adamw_torch</code>
Mixed precision (bf16)	True
Gradient checkpointing	True

Part II: LLM SFT with Q-LORA

Implement Parameter-Efficient Fine-Tuning (PEFT) for skill acquisition using the Qwen model.

Model and Quantization

- **Base Model:** Qwen/Qwen2-1.5B-Instruct (1.5B parameters).
- **Quantization:** Use `BitsAndBytesConfig` for 4-bit Q-LORA, setting `bnb_4bit_quant_type` to "nf4".

Eng. Hana Ghanem
Eng. Mazen Yasser

Eng. Maria Bassem
Eng. Mohamed Salama

booktabs siunitx

SFT Configuration Parameters

Parameter	Value/Setting
LoRA Rank (r)	16
Target Modules	<code>all-linear</code>
Per device train batch size	1
Gradient accumulation steps	4
Number of training epochs	1
Learning rate	2×10^{-4}
Optimizer	<code>paged_adamw_8bit</code>
Maximum sequence length	1024

Eng. Hana Ghanem
Eng. Mazen Yasser

Eng. Maria Bassem
Eng. Mohamed Salama

Part III: Experimental Alignment (DPO)

Use DPO to correct the dangerous compliance learned by the SFT model.

$$\mathcal{L}_{DPO}(\pi_{\theta}; \pi_{ref}) = -E_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right]$$

You can experiment different values for the β parameter and understand how it affect the model alignment process.

booktabs

DPO Configuration Parameters

Parameter	Value/Setting
Per device train batch size	1
Gradient accumulation steps	4
Experimental Parameter (β)	Test: 0.1, 0.5, 0.8, 1.0
Learning rate	1×10^{-5}
Number of training epochs	3
Maximum prompt length	512
Gradient checkpointing	True

Technical Implementation Notes

- **Training on Completion Only (SFT):** For the SFT phase, make sure to compute the loss on the model answers only and not your system prompt or query.
- **Learning Rate Scheduler:** Use a learning rate scheduler of your choice. This is standard practice, warming up the LR initially and cooling it down towards the end of the run for improved stability and convergence, especially for pre-trained LLM.
- **Weights & Biases (wandb):** It is highly recommended to use **wandb** (Weights & Biases) for logging all training runs. Enable this feature in your **TrainingArguments** to track $\mathcal{L}_{SFT}$ and $\mathcal{L}_{DPO}$, which is essential for your analysis. You can use **tensorboard** or **comet** as well.